

Предефиниране на оператори в C++

Семинар 6
Серхан

Оператори като $+$, $==$, $<$, $<<$ и т.н. са само за примитивни типове. За наши класове операторите ги няма по подразбиране.

Предефинирането на оператори позволява да се дефинира поведението на стандартните оператори за нашите класове.

$p1 == p2 \rightarrow p1.operator==(p2) \rightarrow$ метод на класа
 $\rightarrow operator==(p1, p2) \rightarrow$ свободна функция

Синтаксис:

• метод на клас:

```
class MyClass {
```

```
public:
```

```
    ReturnType operator OP (параметри) const {
```

```
        // тяло
```

```
    }
```

↓ символът на оператора

→ този метод не променя $*this$ при оператори за сравнение е задължителен

• свободна функция:

```
    ReturnType operator OP (параметри
```

```
    { // тяло
```

```
    }
```

$(const MyClass \& lhs, const MyClass \& rhs)$

Достъп:

$p1 == p2$

— когато оператор $==$ е метод на класа, $p1$ е $*this$, а $p2$ е $other$; методът има директен достъп до private членове

$p1 == p2$

— когато оператор $==$ е свободна функция, $p1$ е lhs , $p2$ е rhs ; функцията няма достъп до private членове без friend декларация.

Friend - при оператори, които изискват достъп до вътрешното състояние

Оператор:	Форма:
=, [], (), →	Метод
==, <, >, <=, >=, !=	Метод или friend функция
<<, >> ,	Friend функция
+, -, *, /	Friend функция

Приятелски функции и класове (friend)

- friend - декларация в тялото на клас, която дава функция или друг клас достъп до private и protected членовете

```
class Person {
```

```
private:
```

```
    char* name;
```

```
    int age;
```

```
public:
```

```
    friend void printSecret (const Person& p); → декларация
```

```
}
```

```
void printSecret (const Person& p) → дефиниция извън класа
```

```
{ cout << p.name << ", " << p.age << "\n"; → достъп до private
```

```
}
```

- friend - най-често при предефиниране на оператори, при които лявата страна не е обект от класа

```
friend std::ostream& operator<< (std::ostream& os, const Person& p);
```

```
std::ostream& operator<< (std::ostream& os, const Person& p) {
```

```
    os << p.name << "(" << p.age << ")";
```

```
    return os;
```

```
}
```


friend класове - всички методи на приятелския клас имат достъп до private членовете

```
class Engine {  
    int horsepower;  
    friend class Car; → клас Car има достъп до всичко в Engine  
    ...  
}
```

• Когато използваме friend:

- при оператори << и >>
- при оператори, при които двете страни са от размитен тип
- при класове, където достъпът до private е оправдан по дизайн

Задължителни оператори:

• operator == - като метод

```
bool operator==(const Person& other p) const  
{ // тяло }
```

```
bool operator!=(const Person& other p) const - != се извежда от ==  
{ return !(*this == other); }
```

• operator < - като метод

```
bool operator<(const Person& other) const - след него можеш да  
{ // тяло }
```

изберем останалите оператори за сравнение

```
bool operator>(const Person& other) const { return other < *this; }
```

```
bool operator<=(const Person& other) const { return !(other < *this); }
```

```
bool operator>=(const Person& other) const { return !(*this < other); }
```

Оператор $<=>$ - тройно сравнение - един оператор вместо шест (int, char)

`std::strong_ordering` - пълна наредба - равни обекти са идентични

- При $<=>$ се пишат само оператори $<=>$ и $==$ и компилаторът автоматично генерира останалите четири.

```
bool operator == (const Person& other) const  
{ return age == other.age; }
```

```
std::strong_ordering operator <=> (const Person& other) const  
{ return age <=> other.age; }
```

- Ако всички полета на класа поддържат $<=>$, използваме `= default` и компилаторът автоматично генерира всичко.
($==, !=, <, <=, >, >=$)
- При `= default` се сравняват полетата в реда, деклариран в класа
- При ръчна логика на оператора, пишем изрично какъв е типът на връщане
- При `= default`, използваме `auto`
- При клас с `char*` → пишем логиката ръчно!

Auto - казва на компилатора да определи типа, той се определя по време на компилация спрямо стойността въведено от =

- `auto` губи `const` и `&` → за да се запазят, трябва да се напишат
`const auto ca = 5;` → `const int`
`auto& rb = ref;` → `int&`
`const auto& cr = x;` → `const int&`
при четене!

Аритметични оператори:

- оператор $+$ — може като метод, може като friend функция
- оператор $+=$ — като метод, връща $*this$ за верижен запис
- унарен оператор $-$ (отрицание) — като метод
Vector a(1,2)
Vector b = -a // be (-1,-2)
- оператор $[]$ (индексиране) — задължително метод!
- 2 версии — за ^{запис} ~~чтение~~ и четене

- За запис: arr[i]=5

```
int& operator[](int index) { валидиране...  
    return data[index];  
}
```

- За четене от const обект: cout << arr[i];

```
const int& operator[](int index) const  
{ валидиране...  
    return data[index];  
}
```

Auto с operator <=> : - без auto - трябва да знаем точния тип
- с auto - компилаторът определя типа

Operator <=> връща специален обект, който носи информация
за три възможни отношения между двата операнда
навернџи - <, =, >

```
int a=20;
```

```
int b=21;
```

```
auto cmp = a <= b
```

```
if (cmp < 0) → a < b ✓
```

```
if (cmp == 0) → a == b ✗
```

```
if (cmp > 0) → a > b ✗
```

```
if (cmp != 0) → a != b ✓
```

• сравнява се само
с 0

Оператор << - изход

- lhs страна е `std::ostream` ⇒ не е метод на класа,
а е friend функция

- връща `std::ostream&`, за да може вериген запис - `std::cout << p1
<< p2`

Оператор >> - вход

- lhs страна е `std::istream` ⇒ friend функция

```
friend std::istream& operator>>(std::istream& is, Person& p);
```

```
std::istream& operator>>(std::istream& is, Person& p) {
```

```
    char buffer[256];
```

```
    is>>buffer>>ape, p.ape;
```

```
    p.SetName(buffer);
```

```
    return is;
```

```
}
```

`cin >> p;` - гете от
стандартния вход